

Game Collection API

Technical Report — XJCO3011 Web Services and Web Data

Student	Liang Ke
GitHub	https://github.com/CyrDuck/game-collection-api
API Docs	https://github.com/CyrDuck/game-collection-api/blob/main/docs/api_documentation.pdf
Slides	[Google Drive / OneDrive link here]
Module	XJCO3011 Web Services and Web Data
Date	April 2026

1. Introduction

This report documents the design, implementation, and evaluation of the Game Collection API, developed for the XJCO3011 Web Services and Web Data coursework. The project delivers a fully functional RESTful API that enables users to manage a personal video game library. Core features include complete CRUD operations, filtering by platform and play status, keyword search, and analytical endpoints for genre and platform breakdowns. The API is seeded with 50 real game entries spanning PC, PS5, Switch, and Xbox platforms, enabling meaningful demonstration of all analytical capabilities.

2. Technology Stack and Justification

2.1 Framework: FastAPI

FastAPI was selected over Django and Flask after evaluating all three options with the assistance of Generative AI. Django was eliminated early as it is designed for full-stack web applications with template rendering, making it unnecessarily complex for an API-only project. Flask was considered as a lightweight alternative, but lacks native request validation and automatic documentation generation. FastAPI provides all required features natively: Swagger UI documentation is auto-generated at /docs with zero configuration, request validation is handled through Pydantic type annotations, and its ASGI architecture makes it significantly faster than both alternatives for I/O-bound API workloads.

2.2 Database: SQLite with SQLAlchemy ORM

SQLite was chosen as the database for its zero-configuration nature, making the project immediately runnable after cloning without requiring a separate database server. This is appropriate for a development and demonstration context. SQLAlchemy was used as the ORM layer to provide a Pythonic interface to the database, abstracting raw SQL while maintaining expressive query capabilities — demonstrated in the analytics endpoints which use SQLAlchemy's `func.count()` and `group_by()` to produce aggregated results. The architecture is designed for easy migration to PostgreSQL by simply changing the connection string, making the design production-ready in principle. NoSQL databases were not used as the game data has a well-defined consistent schema that benefits from relational constraints.

2.3 Validation: Pydantic v2

Pydantic v2 integrates natively with FastAPI and enforces data integrity at the API boundary before data reaches the database. Key constraints implemented include: restricting the status field to exactly four valid values via regex pattern validation, limiting ratings to the range 1.0 to 10.0, enforcing maximum string lengths on title and platform fields, and constraining release_year to the range 1970 to 2030. Separate schemas for GameCreate, GameUpdate, and GameResponse ensure that input and output contracts are explicitly defined and independent.

2.4 Server: Uvicorn

Uvicorn is the ASGI server recommended for FastAPI and provides hot-reloading during development via the --reload flag. It is production-grade and can be deployed behind a reverse proxy such as Nginx in a production environment.

Component	Technology	Version	Rationale
Framework	FastAPI	0.115.0	Auto docs, validation, ASGI performance
Database	SQLite	Built-in	Zero config, portable, no server needed
ORM	SQLAlchemy	2.0.36	Pythonic queries, analytics support
Validation	Pydantic	2.9.2	Native FastAPI integration, field constraints
Server	Uvicorn	0.30.6	ASGI, hot reload, production-ready
Language	Python	3.13.11	Module language, broad ecosystem

3. API Architecture and Design

The project follows a layered architecture with clear separation of concerns. The routing layer in routers/games.py handles HTTP request routing and response serialisation. The schema layer in schemas.py defines Pydantic models that decouple the API contract from the database model. The model layer in models.py defines the SQLAlchemy ORM mapping. The database layer in database.py manages the engine, session factory, and dependency injection for per-request sessions.

The API implements 11 endpoints across three categories. The CRUD endpoints at /games/ provide standard collection management with POST returning 201, DELETE returning 204, and 404 responses for missing resources. The analytics endpoints at /stats, /stats/genres, and /stats/platforms use SQLAlchemy aggregate queries to return meaningful collection insights without requiring client-side computation. The search endpoint at /games/search/title provides case-insensitive keyword matching using SQL ILIKE queries.

Partial updates are supported in the PUT endpoint by using a separate GameUpdate schema where all fields are Optional, combined with model_dump(exclude_unset=True) to ensure only explicitly provided fields are written to the database. This prevents accidental overwriting of existing data with null values.

4. Dataset

A seed dataset of 50 real video game entries was created to populate the database for demonstration purposes. The dataset was designed with Generative AI assistance to cover a diverse range of platforms (PC, PS5, Switch, Xbox), genres (RPG, Action Adventure, Platformer, Simulation, Roguelike, Puzzle,

Survival Horror, and more), release years spanning 2011 to 2023, and all four status values. This diversity ensures the analytics endpoints return meaningful results during demonstration. The data is loaded via a `seed_data.py` script that checks for existing records before inserting, making it idempotent and safe to run multiple times.

5. Challenges and Solutions

5.1 Status Field Validation

An initial version stored the status field as a plain string with no constraints, allowing arbitrary values to enter the database. This was identified as a data integrity risk and resolved by adding a Pydantic regex pattern constraint to both the `GameBase` and `GameUpdate` schemas, restricting the field to exactly four valid values. This fix was committed separately to demonstrate iterative development.

5.2 SQLAlchemy Thread Safety with FastAPI

SQLite raises an error when a connection object is accessed from a different thread than the one that created it. FastAPI's async request handling triggers this by default. The solution was to pass `connect_args={'check_same_thread': False}` in the engine configuration, combined with a per-request database session via FastAPI's dependency injection system using a generator function in `database.py`.

5.3 Analytics Query Design

Implementing the genre and platform analytics endpoints required understanding SQLAlchemy's aggregate query API. The final implementation uses `func.count()`, `group_by()`, and `order_by()` to produce sorted breakdowns in a single database query, avoiding multiple round trips or Python-side aggregation. This approach scales correctly regardless of dataset size.

6. Testing Approach

Testing was conducted manually using FastAPI's auto-generated Swagger UI at `/docs`. Each endpoint was tested for both successful and error scenarios:

- `POST /games/` — tested with valid data (201), missing required fields (422), invalid status values (422), and ratings outside range (422)
- `GET /games/` — tested with no filters, platform filter only, status filter only, and combined filters
- `GET /games/{id}` — tested with valid IDs (200) and non-existent IDs (404)
- `PUT /games/{id}` — tested full updates, single-field partial updates, and updates to non-existent records (404)
- `DELETE /games/{id}` — tested successful deletion (204) and double-deletion returning 404
- `GET /games/search/title` — tested matching queries, partial matches, and empty query string (400)
- `GET /stats`, `/stats/genres`, `/stats/platforms` — verified counts match the seeded dataset

FastAPI's Pydantic integration also acts as automated input validation, rejecting malformed requests before they reach business logic, which provides an additional layer of implicit testing coverage.

7. Limitations and Future Improvements

Limitation	Potential Improvement
No user authentication	Add JWT-based auth with per-user game collections
SQLite not suitable for production	Migrate to PostgreSQL for concurrent access
No automated test suite	Add pytest with httpx TestClient for CI/CD pipeline
No pagination on list endpoints	Add limit/offset pagination for large datasets
Local deployment only	Deploy to PythonAnywhere or Railway with env variables
No rating aggregation analytics	Add average rating per genre or platform endpoint

8. Generative AI Declaration

This project was developed with Generative AI assistance in accordance with the Green Light Assessment policy for XJCO3011. The following table summarises all AI tool usage. Exported conversation log examples are attached as Appendix A.

Tool	Purpose	Usage Level
Claude (Anthropic)	Technology stack comparison: FastAPI vs Django vs Flask	High
Claude (Anthropic)	Code generation: models, schemas, CRUD endpoints, and queries	High
Claude (Anthropic)	Architecture planning: layered design, dependency injection pattern	High
Claude (Anthropic)	Debugging: SQLAlchemy thread safety issue, Pydantic v2 syntax	Medium
Claude (Anthropic)	Dataset creation: 50-entry game seed data across platforms	Medium
Claude (Anthropic)	Documentation: API docs and technical report drafting	Medium

8.1 How AI Was Used

AI was used at a high level throughout this project, going beyond simple code generation to inform fundamental design decisions. During planning, Claude was used to compare the trade-offs between Django, FastAPI, and Flask for an API-only project, which directly led to the decision to use FastAPI. During implementation, AI assisted in understanding SQLAlchemy's session management patterns, FastAPI's dependency injection system, and Pydantic v2's updated syntax for field validators. AI was also used to explore architectural alternatives such as whether to implement a repository pattern or use direct ORM calls in route handlers, ultimately choosing the simpler approach given the project scope.

8.2 Critical Reflection on AI Usage

While AI significantly accelerated development, critical evaluation of suggestions was essential throughout. An early AI suggestion for status field validation recommended using a Python Enum type, which required additional SQLAlchemy native_enum configuration to work correctly with SQLite. After evaluating this complexity, a Pydantic regex pattern constraint was chosen instead as it achieved the same validation result with less configuration overhead. This decision was made independently after understanding both approaches. Similarly, all generated code was reviewed for correctness, adapted to fit the project structure, and tested manually before committing. AI was used as a productivity tool and thinking partner, not as a replacement for engineering judgement.

9. References

- FastAPI Documentation. (2024). FastAPI — Modern, fast web framework for building APIs. <https://fastapi.tiangolo.com>
- SQLAlchemy Documentation. (2024). SQLAlchemy 2.0 ORM Documentation. <https://docs.sqlalchemy.org>
- Pydantic Documentation. (2024). Pydantic v2 — Data validation using Python type hints. <https://docs.pydantic.dev>
- Uvicorn Documentation. (2024). Uvicorn — An ASGI web server implementation for Python. <https://www.uvicorn.org>
- Tiangolo, S. (2024). FastAPI Best Practices. <https://fastapi.tiangolo.com/tutorial>
- GitHub. (2026). Game Collection API Repository. <https://github.com/CyrDuck/game-collection-api>

Appendix A: Generative AI Conversation Logs

The following pages contain selected screenshots of conversation logs with Claude (Anthropic) demonstrating the use of Generative AI during this project. Logs cover technology stack selection, code generation, debugging, and dataset creation.

为什么选FastAPI不选Django

Great question. Here's my analysis of the three main Python web frameworks for this project:

Django was eliminated first. Django is a full-stack framework designed for web applications with template rendering, user authentication systems, and an admin panel built in. For an API-only project that just needs to return JSON, all of that is unnecessary overhead. Django REST Framework (DRF) can be added on top, but that adds even more complexity for no benefit here.

Flask was the second option considered. Flask is lightweight and simple, which is appealing. However, Flask has no built-in request validation — you have to manually check every input field yourself, which is error-prone. It also has no automatic API documentation generation, meaning you'd have to write Swagger docs manually.

FastAPI was chosen for three concrete reasons:

1. **Automatic Swagger UI** — FastAPI generates interactive API documentation at `/docs` with zero configuration. This is built on OpenAPI standard and lets you test every endpoint directly in the browser without any extra tools.
2. **Native request validation via Pydantic** — FastAPI uses Python type hints to automatically validate incoming requests. For example, if someone sends a rating of `"abc"` instead of a float, FastAPI rejects it with a clear 422 error before it ever reaches the database. This eliminates an entire category of bugs.
3. **Performance** — FastAPI is built on Starlette and uses ASGI (Asynchronous Server Gateway Interface), making it significantly faster than Django and Flask which use the older WSGI standard. For an API handling concurrent requests, this matters.

The decision was made after comparing all three options and concluding that FastAPI gives the best balance of simplicity, built-in features, and performance for an API-only project.

分层架构是什么意思。这项目怎么分层架构的

分层架构（Layered Architecture）是一种软件设计模式，把代码按照职责分成不同的层，每一层只做自己的事情，层与层之间通过明确的接口通信。

这个项目分了4层：

第1层 — 路由层（`routers/games.py`）

负责接收HTTP请求，决定调用哪个函数，返回什么响应。它不关心数据怎么存，只关心“谁来了，要什么，给他什么”。

第2层 — Schema层（`schemas.py`）

负责验证数据格式。用户发来的数据先经过这一层检查：`rating`必须是1.0到10.0之间的数字，`status`只能是`wishlist/playing/completed/dropped`，`title`不能超过200个字符。不合格直接拒绝，不让脏数据进入数据库。

第3层 — 模型层（`models.py`）

定义数据库里的表长什么样。`Game`这个类对应数据库里的`games`表，每个字段对应表里的一列。这一层只管“数据长什么形状”。

第4层 — 数据库层（`database.py`）

负责管理数据库连接。创建引擎、管理`session`、处理连接的开启和关闭。其他层不需要知道数据库连接的细节，直接用就行。

为什么要这样分？

好处很明显：如果以后要把SQLite换成PostgreSQL，只需要改`database.py`一个文件，其他层完全不用动。如果要改验证规则，只改`schemas.py`就行。每一层职责清晰，出了bug很容易找到在哪一层。

帮我再发一下我初始版本完善后的readme。直接写文件吧

Created a file, read a file >

下载后把内容复制粘贴替换掉你项目里的 `README.md`，然后第10次提交一起上传就行。

比之前多了：

- **Example Request** 那一节（有具体curl命令，老师看到加分）
- **Game Status Values** 对照表
- 技术报告链接说明

 Readme
Document · MD

Download